

AI-Powered Network Intrusion Detection & Alert Dashboard

Project Reference & Technical Summary

1. The Big Picture

- **What it is:** A tool that watches network traffic, automatically decides whether each piece of traffic is normal or an attack, and displays alerts on a live dashboard.
 - **What it detects (5 categories):** normal traffic (BENIGN), DDoS floods, port scans, brute force login attempts, and web attacks.
 - **Why it matters:** Networks face constant attacks, and a person cannot watch every connection by hand. Machine learning can spot attack patterns automatically, faster and at larger scale than a human, and a dashboard turns those detections into something an analyst can actually read and act on.
 - **Team roles:** the machine learning model was led by Cole, the dashboard and live demo system by Brian, and the research, attack scenario analysis, documentation, and presentation by Frank.
 - **Main tools used:** Python, Streamlit (the dashboard), Scikit-learn and XGBoost (the models), Plotly (the charts), Scapy (live packet capture), deck.gl (the map), and the CICIDS2017 dataset. The demo runs on VMware virtual machines using Kali Linux and DVWA.
-

2. The Dataset (what the model learns from)

- The project uses **CICIDS2017**, a large public dataset of real network traffic that researchers commonly use to test intrusion detection systems.
- It comes as several daily capture files and is large (close to a gigabyte total).
- Each row is one **network flow**, basically a single connection or session between two machines.
- Each flow is described by dozens of numeric measurements (things like packet counts, how long the connection lasted, and how many bytes per second moved).
- Every flow already has a label saying whether it was normal or a specific kind of attack, which is what lets the model learn the difference.

3. The Machine Learning Model

Cleaning the data

- Removed duplicate rows.
- Replaced invalid/infinite values and dropped rows with missing data, so the model trains on clean, consistent information.

Organizing the labels

- The raw dataset has many specific attack labels, which were grouped into the project's 5 categories.
- Example: the dataset's separate FTP and SSH password-guessing labels were both folded into **Brute Force**, and several different web exploit labels were combined into **Web Attack**. Attack types outside the project's scope were dropped.
- Text labels were converted into numbers, since the models work with numbers, not words.

Training setup

- The data was split **80% for training** (teaching the model) and **20% for testing** (checking it on traffic it had never seen).
- The project started with a simple model that only knew normal vs DDoS. Once that worked, it was expanded to handle all 5 categories.

Comparing models to pick the best one

- **Three different models** were trained and compared so the choice was tested, not assumed:
 - **Logistic Regression:** a simple baseline model.
 - **Random Forest:** many small decision trees that "vote" together on the answer.
 - **XGBoost:** a more advanced model that builds trees in sequence, each one fixing the last one's mistakes.
- Results (Accuracy and F1 Score are both scoring measures where higher is better; F1 balances catching real attacks against raising false alarms):

Model	Accuracy F1 Score	
Random Forest (selected)	99.9856%	99.9856%
XGBoost	99.9946%	99.9945%
Logistic Regression	99.3283%	99.2379%

- All three scored very high. Logistic Regression was the weakest. Random Forest and XGBoost were essentially tied (about 0.009% apart, which is meaningless in practice).
- **Random Forest was chosen** because it is just as accurate as XGBoost, but simpler to run, faster, easier to explain, and already built into the pipeline.
- Performance was strong across all attack types, with slightly weaker results only on Web Attacks because there were fewer examples of those to learn from.

Putting the model to work (the prediction pipeline)

- The trained model was saved to a file so it can be reused anytime without retraining.
- A script takes traffic, runs the model on it, and produces, for each item:
 - the **attack type**,
 - a **confidence** score (how sure the model is),
 - a **severity** level: DDoS and Web Attacks are High, Brute Force is Medium, Port Scans and normal traffic are Low.
- The script exports these results to a file that the dashboard reads.

4. The Dashboard and Live Demo System

The dashboard (the web app everything appears on)

- Built with Streamlit, with a dark "security operations center" look.
- **Live world map:** every detected attack is drawn as a moving arc from the attacker's location to the target, which is pinned at Sacred Heart University.
- **Real-time alert log:** lists each threat with its time, attacker address, target address, attack type, and confidence.
- **Summary cards:** total threats detected, number of attack types seen, number of attacking sources, and average confidence.
- **Charts:** a bar chart of threats by type, a pie chart showing the share of each type, and a timeline of activity over time.
- **Filters and controls:** a sidebar to filter by attack type and set a minimum confidence, plus a button to clear the log and reset for a fresh demo.

Connecting the model to the dashboard

- The dashboard was first wired to read the model's results file so it showed real model output.
- The final version is driven entirely by **real attacks captured live**, with no fake or sample data anywhere.

Listening to live traffic (the capture program)

- A program written with Scapy listens to network traffic on an isolated adapter.
- It only looks at traffic aimed at the target machine, and groups what it sees into rolling 5-second windows.
- It identifies the attack type by its fingerprint:
 - many different destination ports from one source = **Port Scan**
 - a flood of bare connection-request packets = **DDoS**
 - lots of web-port traffic carrying real data = **Web Attack**
 - repeated traffic to the SSH login port = **Brute Force**

- It writes results continuously to a file the map reads, so attacks show up within a couple of seconds of happening.

Building the demo lab

- Two virtual machines were set up on a sealed-off virtual network:
 - **Kali Linux** as the attacker.
 - a **Linux machine** as the target, running an SSH login service and Damn Vulnerable Web Application (DVWA).
- Each attack is launched with a real, standard hacking tool: **nmap** (port scan), **hping3** (DDoS flood), **hydra** (brute force), and **nikto** (web attack). Ordinary traffic stands in for normal/benign activity.
- Keeping everything on an isolated network makes the demo self-contained and keeps the captured data clean.

Keeping the map smooth (a small helper server)

- A tiny local web server lets the live map refresh and save its log on its own.
- This fixed a problem where refreshing the whole page made the map flicker and reset its zoom every few seconds.

5. How the Two Halves Fit Together

- Think of it as two pipelines that meet at the dashboard.
- **The model side (the proof):** runs the trained model on the benchmark dataset to show it is highly accurate on known, labeled data.
- **The live side (the demo):** captures real attacks as they happen, classifies them, and shows them live on the dashboard, proving the system works in real time.
- **Combined story:** an accurate trained model plus a working real-time detection and visualization system.

6. Results and Current Status

- The system detects all 5 attack categories.
- It reaches near-perfect accuracy (about 99.99% with Random Forest) on the benchmark dataset.
- The dashboard is fully built and runs on real captured attacks, not mock data.
- A complete live attack lab exists and all four attack types have been run against the dashboard.
- **Remaining work:** final verification of the live attack demonstrations before the presentation, and adding the model's per-category results into the written report.